

Week 4:

Android

Fundamentals

1. App components

a. Activities

- i. The activity lifecycle
- ii. Override
- iii. Tasks and back stack
- iv. Save UI states

b. Services

- i. Types of services
 - 1. Foreground
 - 2. Background
 - 3. Bound
- ii. Starting and stopping a service
- iii. The service lifecycle

c. Broadcast receivers

- i. Receiving broadcast
- ii. Sending broadcast

d. Content providers

2. Intents and starting activities

- a. Explicit intent
- b. Implicit intent
- c. Pending intent
- d. Intent filter
- e. Passing data between activities

3. Fragments

- a. Fragment manager
- b. Fragment transactions
- c. Add fragment via XML
- d. Add fragment programmatically
- e. Fragment lifecycle

4. More

- a. Context
- b. Serializable
- c. Parcelable
- d. View binding
- e. Data binding
- f. App manifest
- g. XML
- h. XML namespaces
- i. AlertDialog/FragmentManager
- j. PopupMenu
- k. Toast
- l. Snackbar

5. ANRs

- a. Causes of ANR
- b. How to prevent ANR

6. Platform architecture

- a. Linux kernel
- b. Hardware abstraction layer
- c. Android runtime
- d. Native C/C++ libraries
- e. Java API framework
- f. System apps

7. Layouts in views

- a. View
 - i. Button
 - ii. TextView
 - iii. EditText
 - iv. ImageView
 - v. RadioGroup
 - vi. RadioButton
 - vii. Checkbox
- b. ViewGroup
 - i. LinearLayout
 - ii. RelativeLayout
 - iii. ConstraintLayout
 - iv. FrameLayout
 - v. ListView
 - vi. GridView

App Components

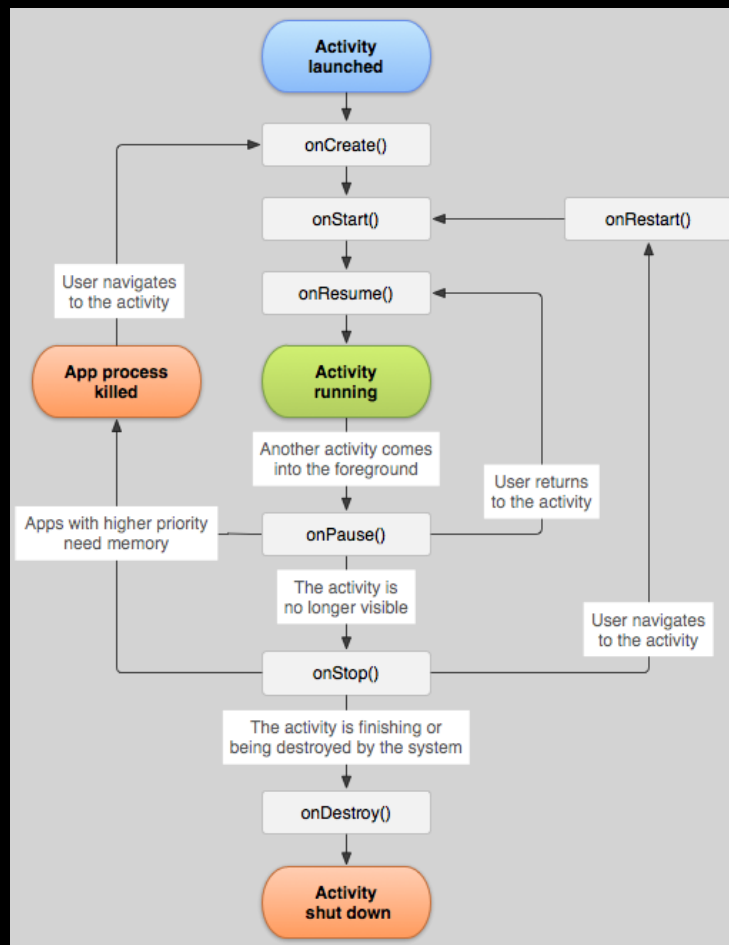
- App components are the essential building blocks of an Android app.
- Each component is an entry point.
 - i. Activities
 - ii. Services
 - iii. Broadcast receivers
 - iv. Content providers

Activities

- Activity represents a single screen with UI.
- Basic building blocks of Android application.
- Each activity has its own lifecycle.
- Activities can also interact with each other and pass data between them.

The activity lifecycle

- Various states an activity goes through from creation destruction.



Override

```
1  override fun onCreate(savedInstanceState: Bundle?) {
2      super.onCreate(savedInstanceState)
3      setContentView(R.layout.activity_main)
4  }
```

Tasks and back stack

- A task is a collection of activities that users interact with when trying to do something in your app.

- These activities are arranged in a stack called the back stack in the order in which each activity is opened.

Save UI states

- `SavedInstanceState` is a mechanism to preserve the state of an activity.
- Allows you to restore data when the activity is recreated.

Services

- A service is an application component that can perform long running operations in the background.
- It does not have a UI.
- It can continue running even after the user switches to another application.

Types of services

i. Foreground

- Performs some operation that is noticeable to the user.
- Foreground services must use notification.
- Continue running even when the user isn't interacting with the app.

ii. Background

- Perform an operation that isn't directly noticed by the user.

iii. Bound

- A service is bound when an app component binds to it by calling *`bindService()`*.
- *Offers client service interface to interact with the service.*
- *Runs only as long as another app component is bound to it.*



```
1 class ExampleService : Service() {
2     private var startMode: Int = 0 // indicates how to behave if the service is killed
3     private var binder: IBinder? = null // interface for clients that bind
4     private var allowRebind: Boolean = false // indicates whether onRebind should be used
5
6     override fun onCreate() {
7         // The service is being created
8     }
9
10    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
11        // The service is starting, due to a call to startService()
12        return startMode
13    }
14
15    override fun onBind(intent: Intent): IBinder? {
16        // A client is binding to the service with bindService()
17        return binder
18    }
19
20    override fun onUnbind(intent: Intent): Boolean {
21        // All clients have unbound with unbindService()
22        return allowRebind
23    }
24
25    override fun onRebind(intent: Intent) {
26        // A client is binding to the service with bindService(),
27        // after onUnbind() has already been called
28    }
29
30    override fun onDestroy() {
31        // The service is no longer used and is being destroyed
32    }
33 }
```



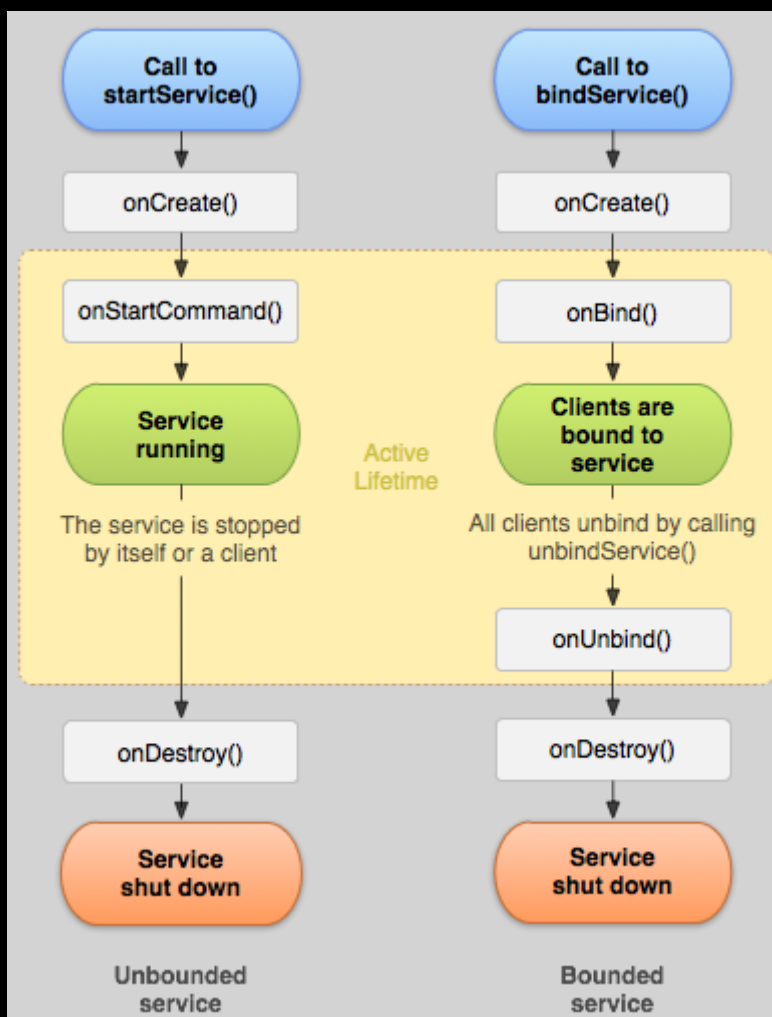
```
1 <manifest ... >
2     ...
3     <application ... >
4         <service android:name=".ExampleService" />
5         ...
6     </application>
7 </manifest>
```

Starting and stopping a service



```
1 // Starting a service
2 startService(Intent(this, HelloService::class.java))
3
4 // Stopping a service
5 stopSelf()
6
7 // Stopping a service from another component
8 stopService(Intent(this, HelloService::class.java))
```

The service lifecycle



Broadcast receivers

- Android apps can send or receive broadcast messages from the Android system and other Android apps.
- These broadcasts are sent when an event of interest occurs.

Receiving broadcasts

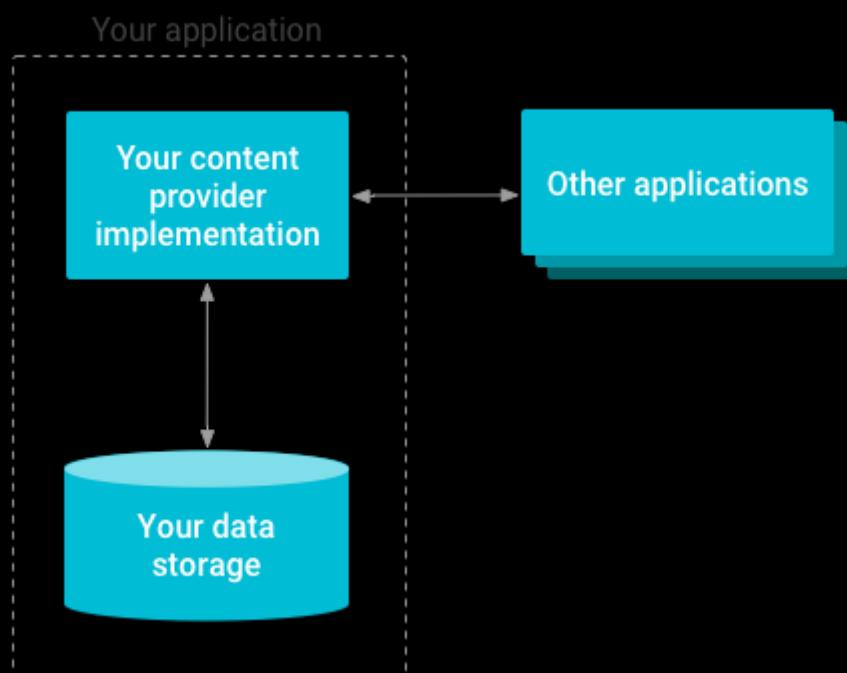
- Apps can receive broadcasts in two ways
 - I. manifest-declared receivers.
 - II. context-registered receivers.

Sending broadcasts

- Android provides three ways for apps to send broadcast
 - I. `sendOrderedBroadcast(Intent, String)`
 - II. `sendBroadcast(Intent)`

Content providers

- Content providers can help an application manage access to data stored by itself or stored by other apps and provide a way to share data with other apps.



Intents and starting activities

- An Intent is a messaging object you can use to request an action from another app component. Such as
 - I. Starting an activity.
 - II. Starting a service.
 - III. Delivering a broadcast.

Explicit intent

- Specify the exact component to start.
- Usually within the same app.



```
1 Intent(this, SecondActivity::class.java).also {  
2     startActivity(it)  
3 }
```

Implicit intent

- Do not name a specific component, but instead declare a general action to perform, which allows a component from another app to handle it.



```
1 val sendIntent = Intent().apply {  
2     action = Intent.ACTION_SEND  
3     putExtra(Intent.EXTRA_TEXT, textMessage)  
4     type = "text/plain"  
5 }  
6  
7 try {  
8     startActivity(sendIntent)  
9 } catch (e: ActivityNotFoundException) {  
10     // Define what your app should do if no activity can handle the intent.  
11 }
```

Pending intent

- Grant permission to a foreign app to use the contained intent as if it were executed from your app's own process.

Intent filter

- An Intent Filter in Android is a component of the Android manifest file that tells the Android system which intents an app component can respond to.

Passing data between activities



```
1 intent.putExtra(Intent.EXTRA_TEXT, textMessage)
2
3 val textMessage = intent.getStringExtra(textMessage)
```

- To pass a data class use `@Serializable` and pass the object.
val data = Intent.getSerializableExtra("key") as Data

Fragments

- A Fragment represents a reusable portion of your app's UI.
- Fragments are lightweight and have their own lifecycle.
- Their lifecycle is closely tied to the host activities lifecycle.

Fragment manager

- Responsible for performing actions on your app's fragments, such as adding, removing, or replacing them and adding them to the back stack.

Fragment transactions

- `FragmentManager` can add, remove, replace, and perform other actions with fragments in response to user interaction.
- Each set of fragment changes that you commit is called a transaction.

Add fragment via xml

class ExampleFragment : Fragment(R.layout.example_fragment)



```
1 <androidx.fragment.app.FragmentContainerView
2     xmlns:android="http://schemas.android.com/apk/res/android"
3     android:id="@+id/fragment_container_view"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:name="com.example.ExampleFragment" />
```

Add a fragment programmatically

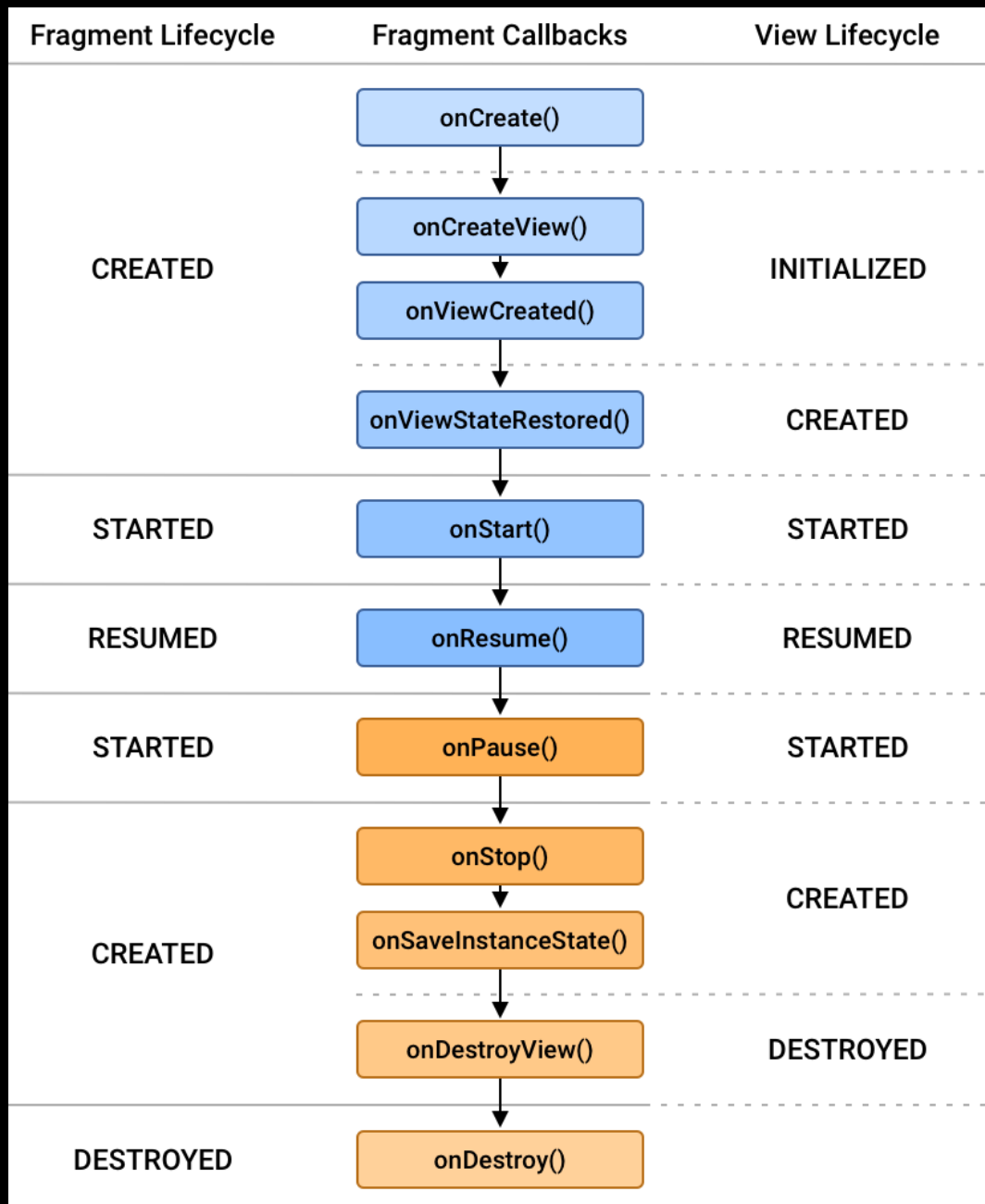


```
1 <androidx.fragment.app.FragmentContainerView
2     xmlns:android="http://schemas.android.com/apk/res/android"
3     android:id="@+id/fragment_container_view"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent" />
```



```
1 supportFragmentManager.commit {
2     setReorderingAllowed(true)
3     add<ExampleFragment>(R.id.fragment_container_view)
4 }
```

Fragment lifecycle



Context

Interface to global information about an application environment. This is an abstract class whose implementation is provided by the Android system. It allows access to application-specific resources and classes, as well as up-calls for application-level operations such as launching activities, broadcasting and receiving intents, etc.

Serializable

Serializable is a standard Java interface used to convert an object into a byte stream, allowing the object to be easily saved to a file, sent over a network, or stored in a database.

Parcelable

Parcelable is an Android-specific interface that enables an object to be passed as a parameter from one activity to another. This is a more efficient method compared to serialization, as it doesn't require the object to be converted into a byte stream.

View binding

It generates a binding class for each XML layout file present in that module. An instance of a binding class contains direct references to all views that have an ID in the corresponding layout.

Data binding

The Data Binding Library is a support library that allows you to bind UI components in your layouts to data sources in your app using a declarative format rather than programmatically.

App manifest

The manifest file describes essential information about your app to the Android build tools, the Android operating system, and Google Play. It declares,

- The components of the app, including all activities, services, broadcast receivers, and content providers.
- The permissions that the app needs in order to access protected parts of the system or other apps.

- The hardware and software features the app requires.

XML

- Extensible Markup Language
- XML is used to define the layout of activities and fragments in Android. Each layout file represents a single screen or part of a screen in the app.
- Basically, the XML tags are not predefined in XML. We need to implement and define the tags in XML.

XML namespaces

XML files in Android often include a namespace declaration like `xmlns:android="http://schemas.android.com/apk/res/android"`, which is necessary to distinguish Android attributes from other XML attributes.

ANRs

- When the UI thread of an Android app is blocked for too long, an "Application Not Responding" (ANR) error is triggered.
- If the app is in the foreground, the system displays a dialog to the user.
- The ANR dialog gives the user the opportunity to force quit the app.

Causes of ANR

1. **Main Thread Blocking:** The most common cause is when the main thread (also known as the UI thread) is blocked by a long-running operation, such as:
 - a. Performing network operations.
 - b. Accessing the file system (reading/writing large files).
 - c. Querying a large database.
 - d. Performing heavy computations.
2. **Deadlocks:** If two or more threads are waiting on each other to release resources, a deadlock occurs, which can lead to ANR if it involves the main thread.
3. **Too Many Broadcast Receivers:** If your app registers too many broadcast receivers and they take too long to execute, this can cause the app to hang and trigger an ANR.
4. **ContentProvider Operations:** Long-running queries or database operations in ContentProvider can also block the main thread and lead to ANRs.

How to prevent ANR

1. **Use Background Threads:** Offload long-running operations to background threads using mechanisms like:
 - a. AsyncTask (deprecated in newer versions of Android)
 - b. HandlerThread
 - c. IntentService (also deprecated, use WorkManager instead)
 - d. Executors
 - e. Coroutines in Kotlin
2. **Use Async I/O Operations:** Always perform I/O operations (like network requests and database queries) asynchronously to avoid blocking the main thread.

3. **Optimize ContentProvider Queries:** Make sure any database queries or other heavy operations performed in ContentProvider are efficient and use background threads where possible.
4. **Limit the Execution Time of Broadcast Receivers:** Keep the execution time of BroadcastReceiver as short as possible. If a longer operation is necessary, start a background service.
5. **Avoid Long Synchronous Operations:** Avoid performing long synchronous operations on the main thread.
6. **Use StrictMode:** Utilize StrictMode during development to detect accidental long-running operations on the main thread.

Detecting and handling ANRs

1. **Logging:** Monitor logs (Logcat) to capture potential ANR traces.
2. **StrictMode:** Enable StrictMode to detect potential slow operations on the main thread during development.
3. **System Traces:** Use Systrace and Android Profiler to analyze app performance and detect slow frames or other performance bottlenecks.

Platform architecture

Linux kernel

- The foundation of the Android platform is the Linux kernel.
- For example, the Android Runtime (ART) relies on the Linux kernel for underlying functionalities such as threading and low-level memory management.

Hardware abstraction layer (HAL)

- The hardware abstraction layer (HAL) provides standard interfaces that expose device hardware capabilities to the higher-level Java API framework.

Android runtime

- For devices running Android version 5.0 (API level 21) or higher, each app runs in its own process and with its own instance of the Android Runtime (ART).
- ART is written to run multiple virtual machines on low-memory devices by executing Dalvik Executable format (DEX) files.

Native C/C++ libraries

- Many core Android system components and services, such as ART and HAL, are built from native code that requires native libraries written in C and C++.
- The Android platform provides Java framework APIs to expose the functionality of some of these native libraries to apps.

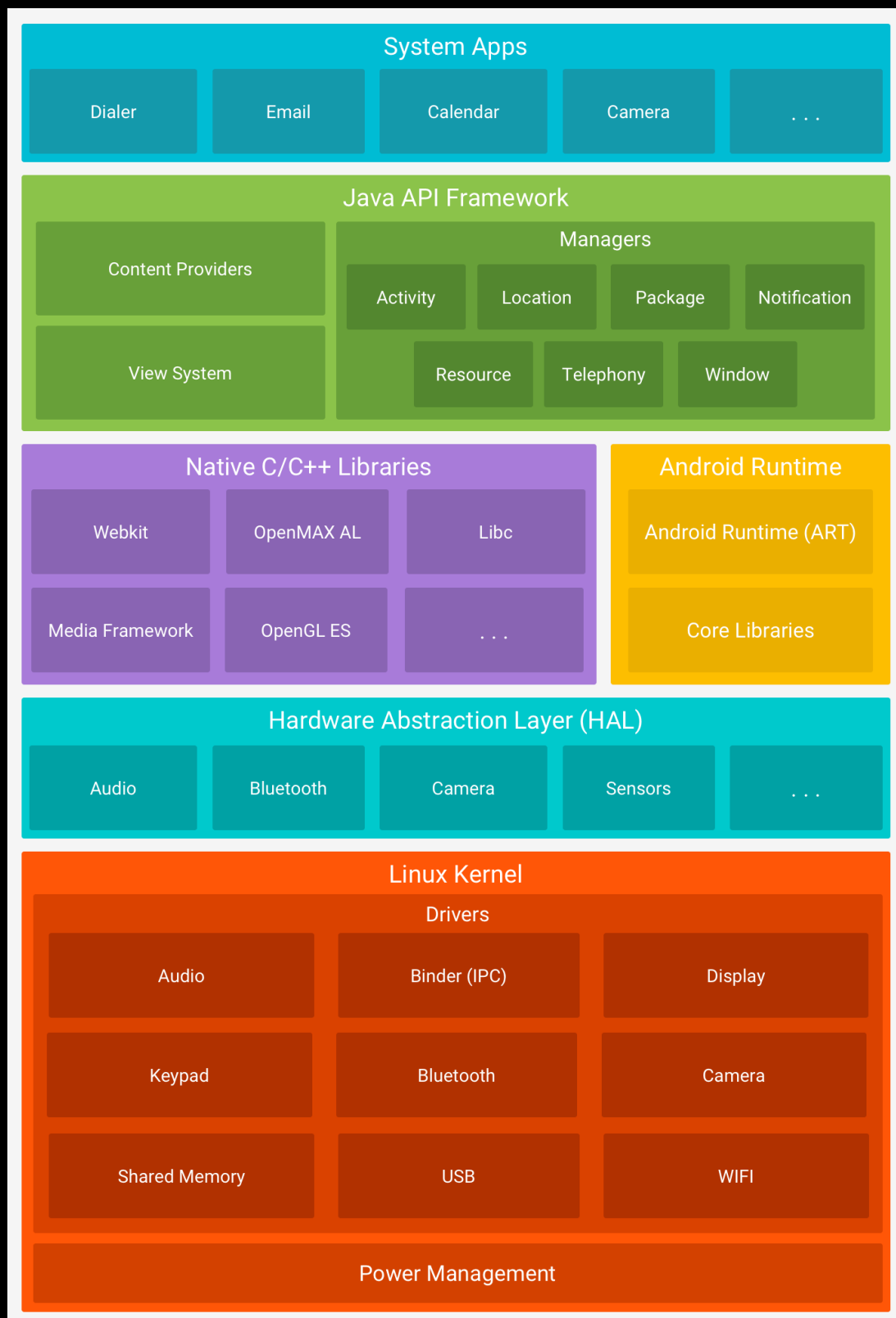
Java API framework

- The entire feature-set of the Android OS is available to you through APIs written in the Java language.
- These APIs form the building blocks you need to create Android apps by simplifying the reuse of core, modular system components and services.

System apps

- Android comes with a set of core apps for email, SMS messaging, calendars, internet browsing, contacts, and more.

- Apps included with the platform have no special status among the apps the user chooses to install.



Layouts in views

- A layout defines the structure for a user interface in your app, such as in an activity.
- All elements in the layout are built using a hierarchy of View and ViewGroup objects.

View

- A View usually draws something the user can see and interact with.
 - I. Button
 - II. TextView
 - III. EditText
 - IV. ImageView
 - V. RadioGroup
 - VI. RadioButton
 - VII. CheckBox

ViewGroup

- A ViewGroup is an invisible container that defines the layout structure for View and other ViewGroup objects.
 - I. **LinearLayout**
 - Aligns all the children either horizontally or vertically.
 - You can specify the layout direction with the android:orientation attribute.
 - II. **RelativeLayout**
 - RelativeLayout is a view group that displays child views in relative positions.
 - The position of each view can be specified as relative to sibling elements (such as to the left-of or below another view) or in positions relative to the parent RelativeLayout area (such as aligned to the bottom, left or center).
 - a. android:layout_alignParentTop
 - b. android:layout_centerVertical
 - c. android:layout_below
 - d. android:layout_toRightOf

III. ConstraintLayout

- ConstraintLayout lets you create large, complex layouts with a flat view hierarchy—no nested view groups.
- It's similar to RelativeLayout, but it's more flexible than RelativeLayout and easier to use with Android Studio's Layout Editor.

IV. FrameLayout

- Contains a single child view.
- It can hold multiple child views, but each child will overlap with others.
- Stack them on top of each other.

V. ListView

- Displays vertically scrollable collection of views.



```
1 val arrayAdapter = ArrayAdapter(context, R.layout.list_item, R.id.listId, array)
2 listView.adapter = arrayAdapter
```

VI. GridView

- Displays items in the grid format, with multiple rows and columns.



```
1 <GridView
2     android:layout_width="match_parent"
3     android:layout_height="match_parent"
4     android:horizontalSpacing="6dp"
5     android:numColumns="2"
6     android:verticalSpacing="6dp" />
```

VII. RecyclerView

- RecyclerView makes it easy to efficiently display large sets of data.
- When an item scrolls off the screen, RecyclerView doesn't destroy its view. Instead, RecyclerView reuses the view for new items.

```
1 class CustomAdapter(private val dataSet: Array<String>) :  
2     RecyclerView.Adapter<CustomAdapter.ViewHolder>() {  
3  
4     class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {  
5         val textView: TextView  
6  
7         init {  
8             textView = view.findViewById(R.id.textView)  
9         }  
10    }  
11  
12    override fun onCreateViewHolder(viewGroup: ViewGroup, viewType: Int): ViewHolder {  
13        val view = LayoutInflater.from(viewGroup.context)  
14            .inflate(R.layout.text_row_item, viewGroup, false)  
15        return ViewHolder(view)  
16    }  
17  
18    override fun onBindViewHolder(viewHolder: ViewHolder, position: Int) {  
19        viewHolder.textView.text = dataSet[position]  
20    }  
21  
22    override fun getItemCount() = dataSet.size  
23 }
```

```
1 val recyclerView: RecyclerView = findViewById(R.id.recycler_view)  
2 recyclerView.layoutManager = LinearLayoutManager(this)  
3 recyclerView.adapter = customAdapter
```